

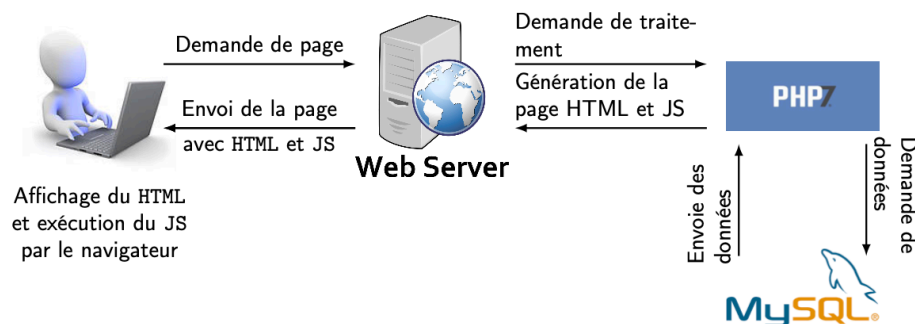
PHP et MySQL

[Avant de commencer essayez d'installer un server web qui tourne PHP - regardez le guide d'installation PHP (cours 4) et essayez de faire l'activité PHP.]

Notre but finale dans ce cours est de créer un site web dynamique. Avec PHP on a déjà commencer à faire des pages web qui affichent des contenus différents sur une même page en fonction de certaines variables (l'heure / date, les paramètres passés dans le URL, etc.). Mais c'est aussi intéressant d'adapter nos pages web selon de contenu / données stockés (ex .info sur preferences d'utilisateur, leur dernière visite, des nouvelles produits dans notre base des données, ...). Un site dynamique va demander donc beaucoup d'informations (images, textes, etc.), qui pose des questions sur : l'organisation de son stockage (chemins d'accès, organisation, etc.), et sur la mise-à-jour de ces informations. Là, nous avons besoin d'un système de gestion de données.

Le MySQL est un système de gestion de bases de données relationnelles (SGBD). Une base de données est un ensemble structuré de données sous forme des tables. Il y a beaucoup des outils pour créer des sites dynamiques (y compris avec des systèmes de gestion des données non-relationnelles). Pour les systèmes BD rationnelles on a déjà pas mal des SGBDs, comme SQLite, MogoDB, PostgreSQL, MariaDB, ... Mais pour commencer, le binôme PHP et MySQL fonctionne très bien : le PHP va faire les calculs / traitement des données pour créer les pages web dynamiquement et le MySQL sera notre bases de données (stockage). En plus, MySQL reste assez simple et fiable, et après si on connait PHP+MySQL le passage à d'autres types de bases de données n'est pas trop compliqué.

Le MySQL (et PHP) existent du coté serveur, voici un schema de regroupement des technologies qu'on a vue jusqu'ici. Côté client, l'accès à la base de données est totalement transparent : tout se passe au niveau du serveur et des scripts en PHP.



Pour connected un SGBD avec PHP nous pouvons soit utiliser un API spécifique comme `mysqli` (il y en a plusieurs !), soit passer par un API indépendante du SGDB comme PDO (PHP Data Object) ou ODBC (Open DataBase Connectivity). Les deux solutions ont leurs supporters et leurs détracteurs. Il n'y a pas de solution complètement idéale.

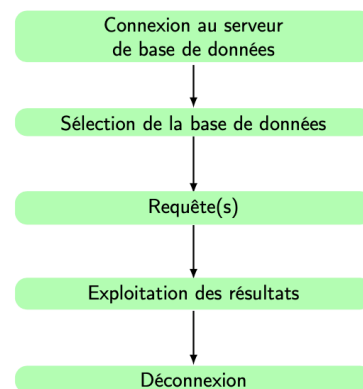
1. Bases de données Relationnelles (rappels)

Mais pourquoi utiliser des bases de données (BD) plutôt qu'un fichier quelconque pour sauvegarder des données ? Bon, les BD ont deux avantages par rapport à d'autres méthodes de stockage : (i) elles peuvent stocker de très grandes quantités de données ; et en plus (ii) elles peuvent faire des requêtes (récupérer) des données de façon simple et rapide. Les bases de données sur lesquelles nous travaillerons reposeront sur le modèle relationnel. C'est le modèle le plus répandu et le plus classique, où les données sont organisées en table (lignes, colonnes). Chacune des colonnes représentant un attribut de données, et chaque attribut est typé. Le langage standard utilisé pour effectuer des requêtes et des mises à jour des données est le SQL (Structured Query Language).

Le SQL est le langage pour accéder aux BD et pour les manipuler. Nous allons l'utiliser à fin d'exécuter toutes sortes de requêtes : récupérer des données, les mettre à jour, insérer ou enlever des données, etc. Le SQL est un langage qui possède sa propre syntaxe et qu'on ne peut pas apprendre en détail ici. Mais nous allons voir des exemples d'utilisation courant pour commencer.

Dans notre cours, nous allons utiliser le système de gestion de bases de données MySQL.

L'interaction entre le code en PHP et la base de données se fera toujours suivant le même déroulement :



Pour le cours, nous ferons le choix d'utiliser l'API `mysqli` pour interagir avec la base de données MySQL. Comme vu précédemment, il est possible également d'utiliser l'API PDO qui est plus générale. Pour une comparaison des 2 méthodes d'accès, cf. [ici](#) et [là](#). Bien que l'approche PDO est plus générale, elle utilise des concepts de la programmation orientée objet (POO) que pour l'instant on ne connaît pas. Si vous êtes à l'aise avec la POO vous pouvez considérer l'utilisation de PDO (voici plus d'info <https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/connexion-base-donnee-mysqli-pdo/>)

L'API `mysqli` propose deux interfaces : une interface procédurale ; et une interface objet. Dans la suite, nous ferons le choix d'utiliser l'interface procédurale.

2. Connection avec la BD

La connexion à la base de données repose sur l'utilisation de la fonction `mysqli_connect()`.

La fonction `mysqli_connect($servername, $username, $password, $base)` renvoie un identifiant de connexion (dans notre cas stocké dans la variable `$bdd`) qui sera nécessaire pour les opérations suivantes. Elle renvoie `FALSE` en cas de problème.

```

<?php
    /* identifiants pour la connexion avec la BDD */
    /* à adapter selon la configuration */
    /* souvent $host="127.0.0.1:3306"; ceci est le meme que localhost,
    mais si on utilise "localhost" MySQL n'utilise pas le bon port de
    communication.*/
    /* L'info sur le port à utiliser avec mysqli est dans http://
    localhost:8080/dashboard/phpinfo.php section mysqli (souvent 3306).
    Pensez à ouvrir le port 3306 dans votre serveur XAMPP */

    $host="servername"; // $host="localhost";
    $user="username";
    $pass="password";
    $base="base";

    /* connexion à la BDD */
    $bdd = mysqli_connect($host,$user,$pass,$base);
    if (!$bdd) {
        die('Echec de connexion au serveur de base de
        données:'.mysqli_connect_error().' '.mysqli_connect_errno());
    }
    echo "connecté à la base de données";
    mysqli_close($bdd);
?>

```

Il est aussi possible d'effectuer la connexion en 2 temps :

- Connexion au serveur : `mysqli_connect($servername, $username, $password);`
- Sélection de la base : `mysqli_select_db($base)` Renvoie TRUE ou FALSE.

La fonction `mysqli_close(identifiant_de_connection)` ferme la connexion avec la BD (dans notre exemple \$bdd est l'identifiante de connexion).

Utilisation du langage SQL dans le PHP

Nous pouvons utiliser SQL à travers l'API spécifique comme mysqli. Par exemple, imaginons que la BD de notre exemple (avec le nom "base") a une table avec le nom Table1. Dans la table Table1 on veut récupérer les 30 premières lignes de valeurs. Nous pouvons alors faire :

```

$query="SELECT * FROM Table1 LIMIT 0,30";
$response=mysqli_query($bdd,$query);

```

La variable \$query ici contient du texte, mais du texte qui est une requête SQL. Le but de ce cours n'est pas d'apprendre du SQL mais vous pouvez installer à Visual Studio Code de packages pour tester la syntaxe SQL, ou vous pouvez tester votre query SQL directement dans votre BD (dans votre serveur > phpMyAdmin > votre table > SQL).

La fonction `mysqli_query()` applique la requête à votre BD à travers l'identification de connexion (dans notre cas `$bdd`). Pour plus d'information sur la fonction `mysqli_query()`, Cf. <https://www.php.net/manual/en/mysqli.query.php>

Il y aura trois types de requêtes :

- Lecture ;
- Modification/insertion ;
- Suppression

Lors de la récupération ou l'insertion de données, il est nécessaire de (i) de préciser le jeu de caractères utilisés ; et (ii) de protéger les caractères spéciaux comme les apostrophes, guillemets, ou encore antislash pour qu'ils ne soient pas interprétés. Par exemple, il faut noter l'utilisation des apostrophes :

```
/* connexion à la BDD */
$bdd = mysqli_connect("localhost","user","password","mydatabase");

/* Fixe le jeu de caractères de la BDD en UTF-8 */
mysqli_set_charset($bdd, "utf8");

/* Protection des ' " etc. */
$nom=mysqli_real_escape_string($bdd,$_POST['nom']);
$prenom=mysqli_real_escape_string($bdd,$_POST['prenom']);
$adresse=mysqli_real_escape_string($bdd,$_POST['adresse']);

/* Insertion d'un nouvel enregistrement dans la table étudiantes */
mysqli_query($bdd,"INSERT INTO table1 ('$nom','$prenom','$adresse')");
```

Ici la variable `$_POST` est une variable prédéfinie qui collecte le résultat de la fonction POST qui est utilisée dans les formulaires HTML (avec la fonction GET). POST collecte et transmet les entrées utilisateur dans un formulaire. Je vous suggère de faire l'activité optionnelle sur les formulaires (Activity-4b-(optional)-PHP-FormulairesHTML.pdf) dans le cours PHP).

Exploitation des résultats

Dans le cas d'une requête de sélection (SELECT), la fonction `mysqli_query()` renvoie le résultat sous la forme d'un objet composé de plusieurs éléments (jeu de résultats). Il est possible de récupérer les résultats sous plusieurs formes à l'aide des fonctions suivantes :

- `mysqli_fetch_array()` : retourne l'enregistrement sous la forme d'un tableau numérique ou associatif.
- `mysqli_fetch_assoc()` : retourne l'enregistrement sous la forme d'un tableau associatif.
- `mysqli_fetch_row()` : retourne l'enregistrement sous la forme d'un tableau numérique.

- `mysqli_fetch_object()` : retourne l'enregistrement sous la forme d'un objet.
- `mysqli_num_rows()` : permet de connaître le nombre d'enregistrements retournés par une requête de sélection.

Par exemple :

```
$reponse=mysqli_query($bdd,"SELECT nom,login FROM etudiant");

/* version mysql_fetch_array */
while ($ligne=mysqli_fetch_assoc($reponse)){
    echo "<li>Nom : ".$ligne['nom'].
        "login : ".$ligne['login']."</li>";
}
/* version mysql_fetch_row */
while ($ligne=mysqli_fetch_row($reponse)){
    echo "<li>Nom : ".$ligne[0].
        "login : ".$ligne[1]."</li>";
}
/* version mysql_fetch_object */
while ($ligne=mysqli_fetch_object($reponse)){
    echo "<li>Nom : ".$ligne->nom."login : ".
        $ligne->login."</li>";
}
```

Depuis la version 5.4 de PHP, il est possible d'utiliser directement la commande `foreach` pour parcourir le jeu de résultats.

```
$reponse=mysqli_query($bdd,"SELECT nom,login FROM etudiant");
if($reponse) {
    echo "<ul>";
    foreach($reponse as $enr){
        echo "<li>".$enr['nom']." avec pour login ".
            $enr['login']."</li>";
    }
    echo "</ul>";
}
```

Libération de la mémoire et Déconnexion

Quand la variable utilisée pour la réponse à une requête n'est plus nécessaire, il est important de libérer la mémoire utilisée à l'aide de la commande `mysqli_free_result()` :

```
/* libération la mémoire utilisée par les réponses */  
mysqli_free_result ($reponse);
```

La connexion à un serveur de base de données est fermée automatiquement à la fin du script PHP. Il est, cependant, possible de le faire manuellement à tout moment à l'aide de la fonction `mysqli_close()` :

```
/* déconnexion de la base */  
mysqli_close ($bdd);
```

À partir d'ici une référence de PHP+SQL.

Fonctions utiles de l'API mysqli

<code>mysqli_affected_row()</code>	Nombre d'enregistrements concernés par la dernière requête.
<code>mysqli_close()</code>	Déconnexion.
<code>mysqli_connect()</code>	Connexion à un serveur.
<code>mysqli_data_seek()</code>	Déplace le pointeur interne de résultat.
<code>mysqli_query()</code>	Exécution d'une requête.
<code>mysqli_error()</code>	Retourne le texte associé à l'erreur provoqué par la dernière requête.
<code>mysqli_escape_string()</code>	Protège une chaîne pour la passer à <code>mysqli_query()</code> .

<code>mysqli_fetch_array()</code>	Lit une ligne de résultat dans tableau.
<code>mysqli_fetch_assoc()</code>	Idem, dans un tableau associatif.
<code>mysqli_fetch_object()</code>	Idem, dans un objet.
<code>mysqli_fetch_row()</code>	Idem, dans un tableau numérique.
<code>mysqli_free_result()</code>	Efface le résultat de la mémoire.
<code>mysqli_insert_id()</code>	Retourne l'identifiant de la dernière requête INSERT.
<code>mysqli_num_fields()</code>	Retourne le nombre de champs d'un résultat.
<code>mysqli_fetch_fields()</code>	Retourne le prochain champs dans le résultat.
<code>mysqli_num_rows()</code>	Retourne le nombre de lignes d'un resultat SELECT.
<code>mysqli_select_db()</code>	Selectionne une base de données.

Pour une liste complète des fonctions :

Cf. <https://www.php.net/manual/en/mysqli.summary.php>

Liens utiles

1. PHP tuto par l'équipe de documentation PHP <https://www.php.net/manual/en/index.php>
2. Tuto interactive du w3schools <https://www.w3schools.com/php/>
3. En Français. Cours de P.Giraud sur PHP et SQL : <https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/>
4. Internet ...